

COMICCRAFT - AI

COMIC STORY CREATOR USING GEMINI MODELS

PHASE 1 TO PHASE 8

PHASE 1 – BRAINSTORMING AND IDEATION

1.1 Introduction

The Brainstorming and Ideation phase is the first stage of the ComicCraft AI project. The main purpose of this phase is to identify a useful problem and develop an initial idea for solving it using Artificial Intelligence.

The idea for ComicCraft AI was developed to make comic creation easier for users. Creating a complete comic manually involves story writing, character planning, scene development, image creation, panel arrangement, and preparation of the final comic document.

ComicCraft AI is proposed as an AI-based application that helps users transform a simple story idea or prompt into a structured comic.

1.2 Problem Identification

Traditional comic creation involves several separate activities:

- Writing the story
- Developing characters and scenes
- Dividing the story into panels
- Creating suitable illustrations
- Adding narration and dialogue
- Arranging the panels
- Preparing the final comic document

For beginners and users without drawing or storytelling experience, completing these activities manually can be difficult and time-consuming.

Therefore, the project focuses on combining these activities into a single AI-assisted application.

1.3 Project Idea

The proposed project is **ComicCraft AI**, an AI-powered application for generating comic stories and illustrations.

The user provides a story idea or prompt along with additional preferences. The system processes the input and generates structured comic content, illustrations, a comic preview, and an exportable PDF.

The basic concept is:

User Prompt

Story Generation

Panel Creation

Image Generation

Comic Layout

Preview / Export

1.4 Brainstorming

During brainstorming, the following ideas were considered:

- AI-based story generation
- Automatic comic panel generation
- AI image generation
- Character and scene visualization
- Automatic narration and dialogue generation
- Comic preview
- PDF export
- Simple user interface

These ideas were combined to form the final project concept, **ComicCraft AI**.

1.5 Objectives

The main objectives of ComicCraft AI are:

1. To develop an AI-based comic generation application.
2. To convert a simple story prompt into structured comic content.
3. To generate comic panels automatically.
4. To generate suitable illustrations for the comic scenes.
5. To provide narration and character dialogue.
6. To arrange generated content into a structured comic layout.
7. To provide a preview of the generated comic.
8. To allow the completed comic to be exported as a PDF.
9. To reduce the time and effort required for manual comic creation.
10. To demonstrate a practical application of Generative AI.

1.6 Target Users

ComicCraft AI can be useful for:

- Students
- Beginners interested in comic creation
- Story writers
- Creative users
- Teachers and educators
- Content creators
- Hobbyists

1.7 Expected Outcome

The expected outcome of this phase is a clear project idea and a defined objective for developing ComicCraft AI.

The proposed application should allow a user to enter a story idea and receive an AI-generated comic containing story content, panels, illustrations, and a final output.

1.8 Conclusion

The Brainstorming and Ideation phase helped identify the problem, develop the project idea, define the objectives, and determine the expected outcome.

The finalized concept is taken forward to **Phase 2 – Requirement Analysis**.

PHASE 2 – REQUIREMENT ANALYSIS

2.1 Introduction

Requirement Analysis identifies what is required to develop and operate the ComicCraft AI application.

The requirements are divided into functional requirements, non-functional requirements, hardware requirements, software requirements, user requirements, and input/output requirements.

2.2 Functional Requirements

Functional requirements describe the operations that the system should perform.

2.2.1 User Input

The application should allow the user to provide:

- Story Prompt
- Main Character Name
- Setting
- Story Tone
- Art Style

The information entered by the user becomes the basis for comic generation.

2.2.2 Story Generation

The system should generate a structured comic outline from the user's story prompt.

The project uses a five-panel outline approach, where each panel contains information such as:

- Panel number
- Panel title
- Scene description
- Image generation prompt

2.2.3 Narration and Dialogue Generation

The system should expand the generated panel outline into comic-style narration and character dialogue.

2.2.4 Illustration Generation

The system should generate comic-style illustrations based on the image prompts associated with the panels.

2.2.5 Comic Layout

The generated images and story content should be organized into a structured panel-by-panel comic layout.

2.2.6 Comic Preview

The system should display the generated comic before final export.

The preview can contain:

- Panel number
- Panel title
- Comic image
- Scene description
- Caption
- Narration
- Dialogue or related story content

2.2.7 PDF Export

The system should provide an option to export the completed comic as a multi-page PDF.

2.2.8 Error Handling

The application should handle incomplete or invalid inputs and should provide an appropriate response when generation or API processing fails.

2.3 Non-Functional Requirements

2.3.1 Usability

The application should provide a simple interface that can be understood by users without advanced technical knowledge.

2.3.2 Performance

The application should process user requests and generate the required output within a reasonable amount of time, depending on the AI model and image generation process.

2.3.3 Reliability

The application should provide consistent processing from user input to generated output.

2.3.4 Maintainability

The project should be organized into separate modules so that individual components can be modified or maintained easily.

2.3.5 Compatibility

The application should run in a suitable Python development environment and be accessible through a web browser during local deployment.

2.4 Hardware Requirements

The project requires a computer system capable of running Python, the FastAPI server, required AI libraries, and image-generation components.

The system should have:

- Computer/Laptop
- Sufficient RAM
- Sufficient storage
- Internet connection for required API/model access

2.5 Software Requirements

The project uses the following software and technologies:

- Python
- FastAPI
- Uvicorn
- HTML
- CSS
- Jinja2
- Google Gemini API
- Hugging Face Diffusers
- Stable Diffusion
- PyTorch
- Pillow
- FPDF
- Git
- Visual Studio Code

2.6 User Requirements

The user should be able to:

1. Open the ComicCraft AI application.
2. Enter a story idea.
3. Provide character and story preferences.
4. Submit the request.
5. View the generated comic.
6. Review the generated panels.
7. Export the completed comic.

2.7 Input Requirements

The main inputs are:

- Story prompt
- Character name
- Setting
- Tone
- Art style

2.8 Output Requirements

The expected outputs include:

- Structured comic outline
- Narration
- Character dialogue
- Scene descriptions
- Generated illustrations
- Comic preview
- Exportable PDF

2.9 Conclusion

The Requirement Analysis phase defines what ComicCraft AI must provide and what resources are needed to operate the application.

These requirements are used as the foundation for the technical design in **Phase 3 – Project Design**.

PHASE 3 – PROJECT DESIGN

3.1 Introduction

The Project Design phase converts the requirements into a technical structure for ComicCraft AI.

This phase defines the architecture, frontend design, backend design, AI integration, modules, workflow, data flow, and output design.

3.2 System Architecture

ComicCraft AI is organized into the following major components:

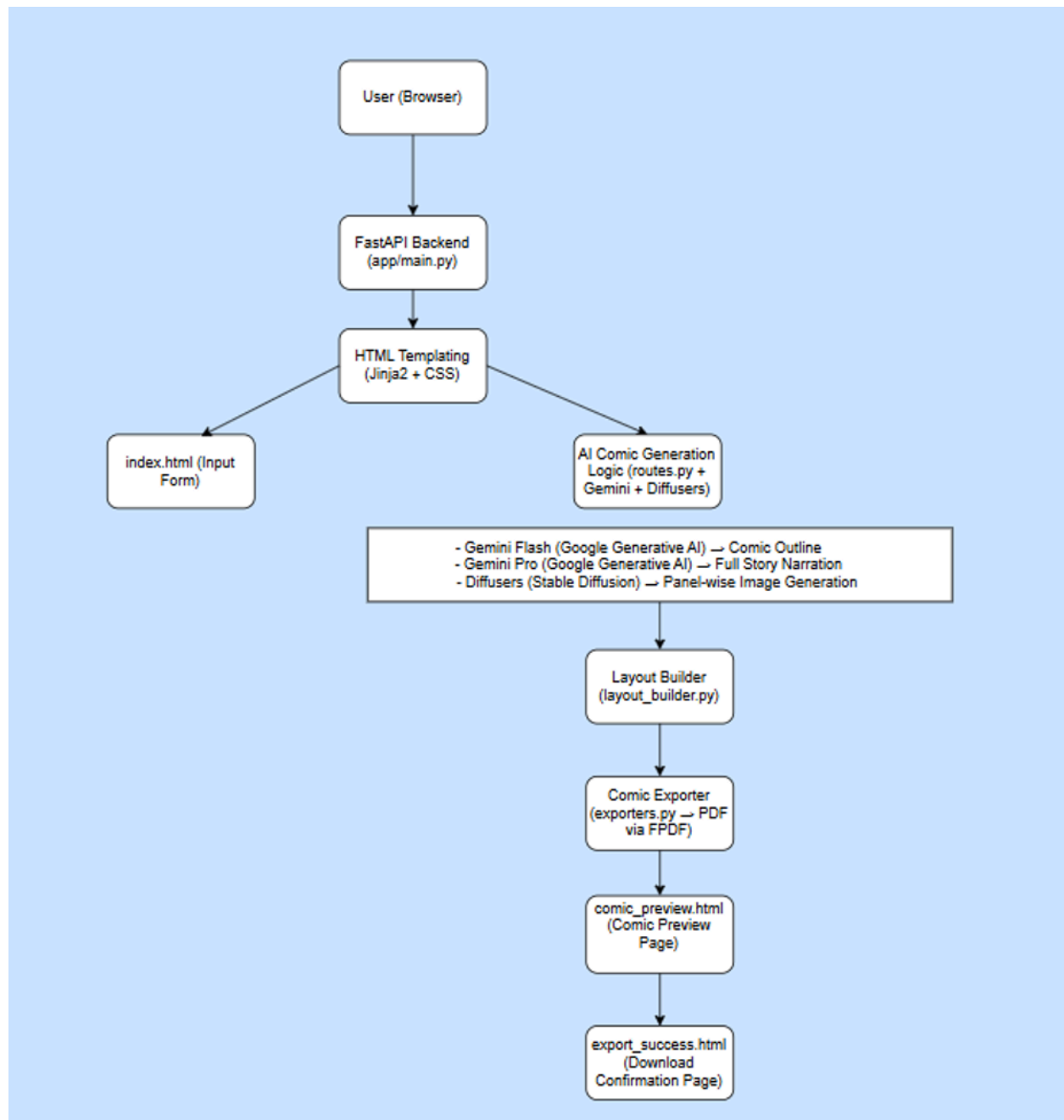
1. Frontend
2. FastAPI Backend
3. AI Generation Services
4. Comic Layout
5. Preview and Export

The frontend collects user information and displays the generated result.

The FastAPI backend manages the application flow and connects the user interface with the generation modules.

The AI services generate the story and illustrations.

The layout and export components prepare the final comic.



3.3 Frontend Design

The frontend is developed using HTML, CSS, and Jinja2 templates.

The main input fields include:

- Story Prompt
- Main Character

- Setting
- Story Tone
- Art Style

The main templates include:

index.html

Used to collect user input.

comic_preview.html

Used to display the generated comic panels and their content.

export_success.html

Used to display confirmation after comic export.

The frontend is connected with FastAPI using Jinja2 dynamic templates.

3.4 Backend Design

FastAPI is used as the backend framework.

The backend is responsible for:

- Receiving user input
- Processing requests
- Calling the required AI generation functions
- Managing the comic-generation workflow
- Building the comic layout
- Handling export
- Rendering the output through templates

The main routing logic is handled through `routes.py`.

3.5 AI Integration Design

The project uses separate AI components for different generation tasks.

Gemini Flash

Used for generating the structured comic panel outline.

Gemini Pro

Used for expanding the panel outline into detailed narration and character dialogue.

Stable Diffusion

Used for generating comic-style illustrations from image prompts.

[Insert AI Integration Diagram Here]

3.6 Module Design

3.6.1 Outline Generation Module

Function: `generate_outline()`

File: `gemini_flash.py`

Generates the structured five-panel comic outline.

3.6.2 Story Generation Module

Function: `generate_story()`

File: `gemini_pro.py`

Expands the panel outline into narration and character dialogue.

3.6.3 Image Generation Module

Function: `generate_image()`

File: `image_generator.py`

Generates comic-style illustrations and saves the generated images.

3.6.4 Layout Module

Function: `build_comic_layout()`

File: `layout_builder.py`

Combines the generated images with the corresponding panel content.

3.6.5 PDF Export Module

Function: `save_pdf()`

File: `exporters.py`

Compiles the comic panels and content into a multi-page PDF.

```
comiccraft/
|
├─ app/
|   ├── main.py
|   ├── routes.py
|   ├── gemini_flash.py
|   ├── gemini_pro.py
|   ├── image_generator.py
|   ├── layout_builder.py
|   └─ exporters.py
|
├─ templates/
|   ├── index.html
|   ├── comic_preview.html
|   └─ export_success.html
|
├─ static/
|   ├── panels/           # Generated panel images
|   ├── exports/          # Exported comic PDFs
|   └─ fonts/             # Fonts for PDF generation
|
├─ .env                   # Environment variables (API keys)
└─ requirements.txt       # Python dependencies list
```

Structure of Project

3.7 Project Workflow

The designed workflow is:

1. User enters comic details.
2. Frontend sends the information to the backend.
3. Backend processes the request.
4. Gemini Flash creates the panel outline.
5. Gemini Pro generates narration and dialogue.
6. Image generation creates illustrations.
7. Layout builder organizes the panels.
8. Comic preview displays the generated result.
9. PDF export prepares the final document.

3.8 Data Flow

The data moves through the system as follows:

User Input → FastAPI Backend → AI Generation → Panel Content → Images → Layout → Preview → PDF Export

The backend receives the input and sends the relevant information to the appropriate generation modules. The generated content is then combined into the final comic layout.

3.9 Interface Design

The interface is designed to provide a simple comic-creation workflow.

The user enters the required information and submits the form. After generation, the result page presents the comic panels with their visual and textual content.

3.10 Output Design

The final comic output consists of panel-based content containing:

- Panel title
- Generated image
- Scene description
- Caption
- Narration
- Dialogue where applicable

The generated content can then be compiled into a PDF.

3.11 Conclusion

The Project Design phase establishes the technical structure of ComicCraft AI.

It provides the foundation for implementing the application in the next phase, **Phase 4 – Project Planning**.

PHASE 4 – PROJECT PLANNING

4.1 Introduction

The Project Planning phase defines how the ComicCraft AI project will be developed and completed.

The development is divided into milestones and activities so that the project can be implemented in a systematic sequence.

The planning follows five major milestones:

1. Model Selection and Architecture
2. Core Functionalities Development
3. `routes.py` Development
4. Frontend Development
5. Deployment

4.2 Development Sequence

The planned development sequence is:

Model Selection → Architecture → Core Development → Backend Routing → Frontend → Deployment → Testing

Each stage depends on the completion of the required previous activities.

4.3 Milestone 1 – Model Selection and Architecture

Activity 1.1 – Research and Select the Appropriate Generative AI Model

The project requires AI models for:

- Structured comic outlines
- Comic narration
- Character dialogues
- Visual illustrations

The selected AI components are based on their suitability for these tasks.

Activity 1.2 – Define the Application Architecture

The overall application structure is planned by separating:

- User interface
- Backend processing
- AI generation
- Layout generation
- Export

Activity 1.3 – Set Up the Development Environment

The development environment is prepared with Python, a virtual environment, required libraries, and the project directory structure.

The project documentation specifies installation of required libraries and running the application through Uvicorn.

4.4 Milestone 2 – Core Functionalities Development

Activity 2.1 – Develop Core Functionalities

The main generation functions are developed for:

- Comic outline generation
- Story narration and dialogue
- Illustration generation
- Layout creation
- PDF export

Activity 2.2 – Implement FastAPI Backend

The backend is planned to receive user input, process requests, call the required functions, and return the generated result.

The `/generate` flow connects the generation functions into the complete comic creation process.

4.5 Milestone 3 – routes.py Development

Activity 3.1 – Develop Main Application Logic

The main application logic is organized in `routes.py`.

The routing layer connects the frontend requests with the required backend processing and generation functions.

4.6 Milestone 4 – Frontend Development

Activity 4.1 – Design and Develop the User Interface

The interface is planned to provide a simple form for entering comic requirements.

Activity 4.2 – Create Dynamic Templates with Jinja2

Jinja2 templates are used to connect generated backend data with the web pages.

The templates display user input, generated panels, images, narration, and export confirmation.

4.7 Milestone 5 – Deployment

Activity 5.1 – Prepare the Application for Local Deployment

The application is prepared using the required environment, dependencies, static files, templates, and configuration.

Sensitive API credentials are planned to be stored using environment variables rather than directly inside application code.

Activity 5.2 – Test and Verify Local Deployment

The application is started using the FastAPI/Uvicorn development server.

The local application can be accessed through the browser, and the FastAPI documentation can be used for API testing.

4.8 Resource Planning

The project requires:

- Development computer
- Internet connection
- Python environment
- AI API access
- Required software libraries
- Source-code repository
- Project documentation
- Testing environment

4.9 Risk Planning

Possible project risks include:

- AI API connection errors
- Image generation failures
- Incomplete user input
- Dependency installation problems
- Slow generation time
- Insufficient system resources
- Incorrect generated content

These risks are handled through validation, testing, error handling, and separate module design.

4.10 Expected Outcome

The planning phase provides a clear sequence for developing the complete ComicCraft AI application from initial setup through local deployment.

4.11 Conclusion

The Project Planning phase organizes the development work into manageable milestones and activities.

The planned activities provide the roadmap for **Phase 5 – Project Development**.

PHASE 5 – PROJECT DEVELOPMENT

5.1 Introduction

The Project Development phase is where the planned design is implemented as a working ComicCraft AI application.

The development follows the milestones defined in the Project Planning phase.

5.2 Development Environment Setup

The project is developed using Python and a suitable development environment.

A virtual environment is used to keep project dependencies organized.

The required packages include components for:

- FastAPI
- Uvicorn
- Jinja2
- Google Generative AI
- Hugging Face Diffusers
- Transformers
- FPDF
- Pillow
- Accelerate

5.3 Core Functionality Development

5.3.1 Comic Outline Generation

The `generate_outline()` function in `gemini_flash.py` generates a structured five-panel outline.

Each panel contains relevant information such as the panel number, title, scene description, and image prompt.

5.3.2 Story Generation

The `generate_story()` function in `gemini_pro.py` expands the outline into comic-style narration and character dialogue.

5.3.3 Image Generation

The `generate_image()` function in `image_generator.py` generates illustrations based on the image prompts.

The generated images are stored in the project's panel directory.

5.3.4 Comic Layout

The `build_comic_layout()` function in `layout_builder.py` matches the generated images with their corresponding panel content.

5.3.5 PDF Export

The `save_pdf()` function in `exporters.py` compiles the generated comic content into a multi-page PDF.

5.4 Backend Development

The FastAPI backend manages the complete generation process.

The `/generate` route receives form data including:

- Story prompt
- Main character
- Setting
- Tone
- Art style

The backend then performs the required generation operations and prepares the result for display.

The project also includes JSON-based comic generation and supporting routes for image testing and export confirmation.

5.5 Frontend Development

The frontend is connected to the backend using Jinja2.

Homepage

The homepage collects the user's comic requirements.

Comic Preview Page

The preview page displays the generated panels sequentially with their images and textual content.

Export Success Page

The export success page confirms that the comic has been successfully prepared for download.

The source project describes the preview page as combining the generated image, panel title, scene description, caption, narration, and related content.

5.6 Complete Development Workflow

The implemented workflow is:

User Input

↓

FastAPI Request Processing

↓

Gemini Flash – Panel Outline

↓

Gemini Pro – Narration and Dialogue

↓

Stable Diffusion – Illustrations

↓

Layout Builder – Panel Arrangement

↓

Comic Preview

↓

PDF Export

This creates an end-to-end comic generation process.

5.7 Development Result

The development phase produces a working application capable of accepting a user's comic idea and processing it through the AI-based comic-generation pipeline.

5.8 Conclusion

The Project Development phase converts the planned design into an implemented application.

After development, the application proceeds to **Phase 6 – Project Testing**.

PHASE 6 – PROJECT TESTING

6.1 Introduction

The Project Testing phase verifies whether ComicCraft AI works correctly according to the planned requirements.

Testing is performed on the input, API processing, user interface, generated output, and final comic result.

6.2 Testing Objectives

The main objectives are:

1. To verify that valid prompts generate output.
2. To check incomplete or empty inputs.
3. To verify API request handling.
4. To check the display of generated results.
5. To verify basic user-interface usability.
6. To check the readability and relevance of the generated output.
7. To verify the comic generation workflow.

These testing activities are also included in the project's presentation documentation.

6.3 Test Case 1 – Valid Story Prompt

Input: A complete comic/story idea.

Expected Result: The system should process the prompt and generate comic content.

Purpose: To verify the normal generation workflow.

6.4 Test Case 2 – Empty or Incomplete Input

Input: Empty or incomplete required information.

Expected Result: The system should handle the input appropriately rather than processing invalid information without control.

Purpose: To verify input handling.

6.5 Test Case 3 – API Request Handling

Input: User submits a generation request.

Expected Result: The request should reach the FastAPI backend and the required processing should be initiated.

Purpose: To verify communication between the frontend and backend.

6.6 Test Case 4 – Generated Output Display

Input: Successfully generated comic content.

Expected Result: The generated result should be displayed on the comic preview page.

Purpose: To verify frontend rendering.

6.7 Test Case 5 – Image Generation

Input: Valid image prompts generated from the comic outline.

Expected Result: Comic-style illustrations should be generated and associated with the appropriate panels.

Purpose: To verify the image-generation stage.

6.8 Test Case 6 – Comic Layout

Input: Generated images and story content.

Expected Result: Images and corresponding story content should be organized into the correct panel structure.

Purpose: To verify layout processing.

6.9 Test Case 7 – PDF Export

Input: Completed comic layout.

Expected Result: The comic should be compiled into a PDF document.

Purpose: To verify final export functionality.

6.10 Test Case 8 – Output Quality

Input: Successfully generated comic.

Expected Result: The final output should be readable, understandable, and relevant to the user's prompt.

Purpose: To verify the quality of the final result.

6.11 Testing Through FastAPI Documentation

FastAPI provides an interactive API documentation interface that can be used to test the available endpoints during local development.

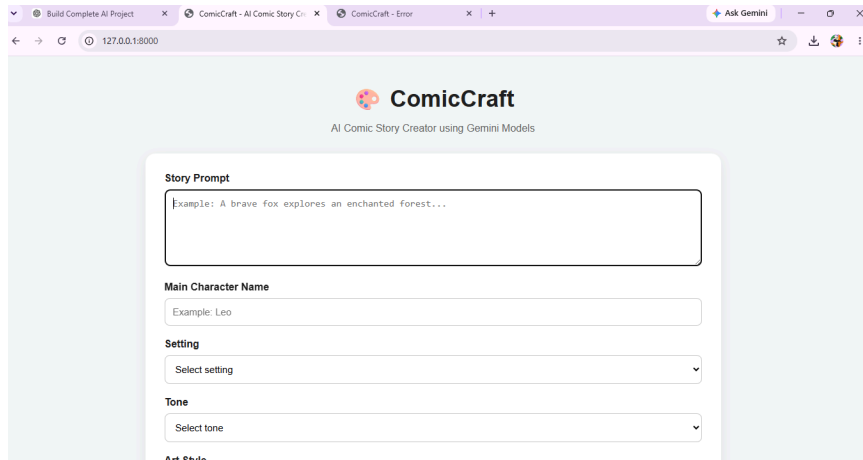
The project documentation identifies endpoints such as:

- `/generate`
- `/generate-comic/json`
- `/export-success`
- `/test-image`

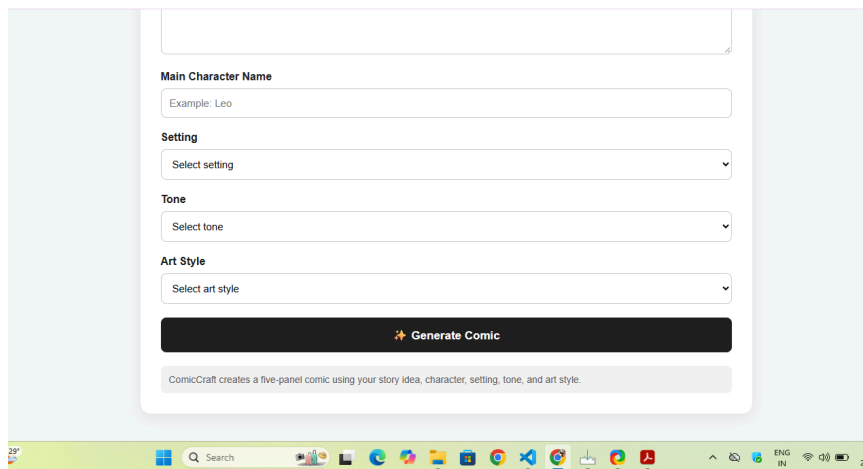
6.12 Testing Result

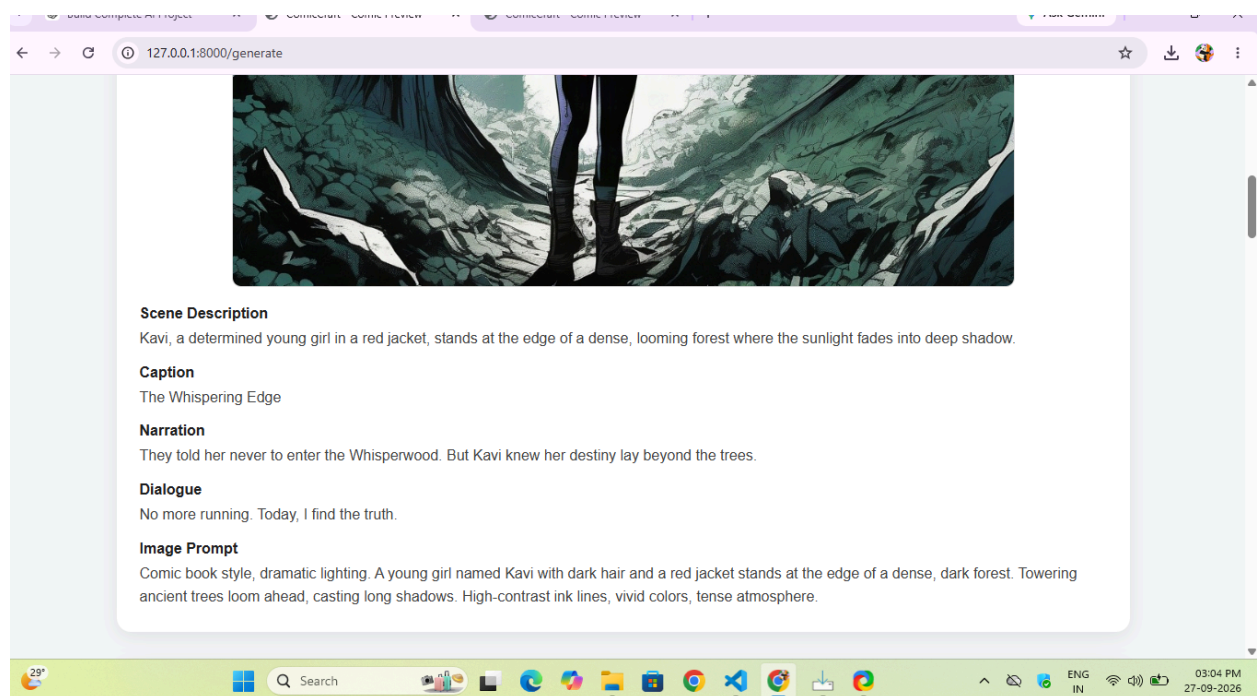
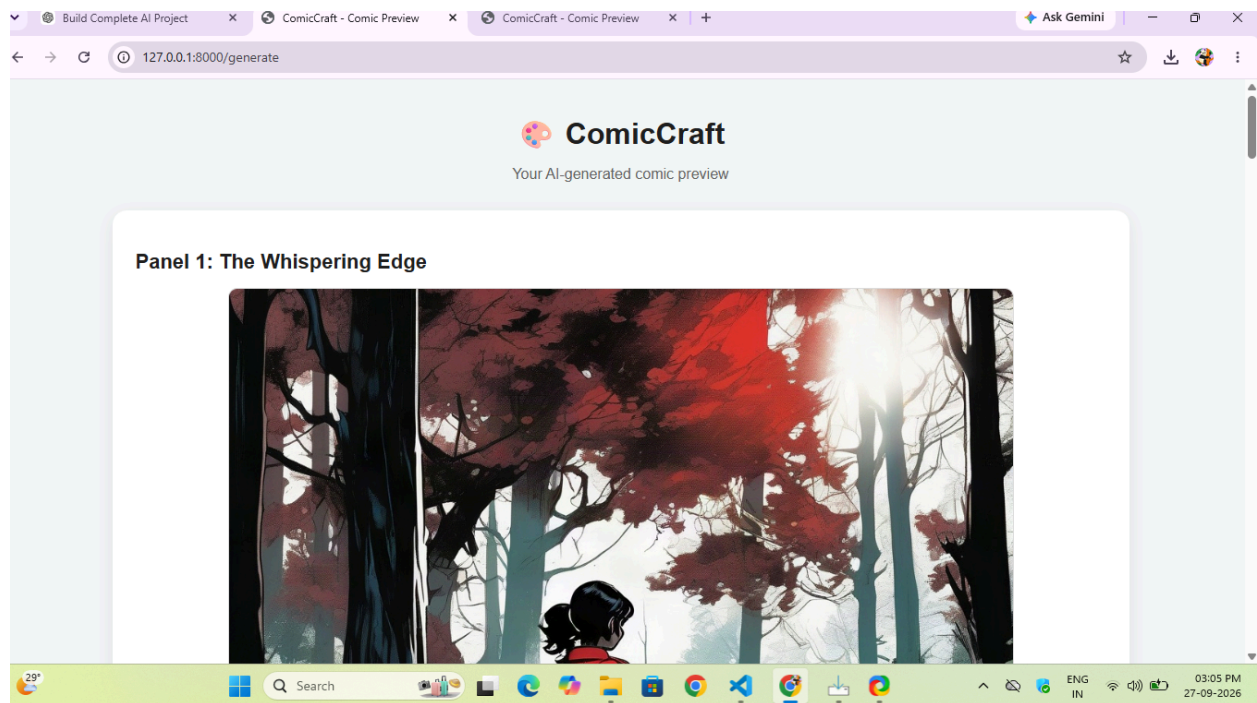
The testing phase is used to identify problems in input processing, API communication, AI generation, image generation, interface rendering, and export.

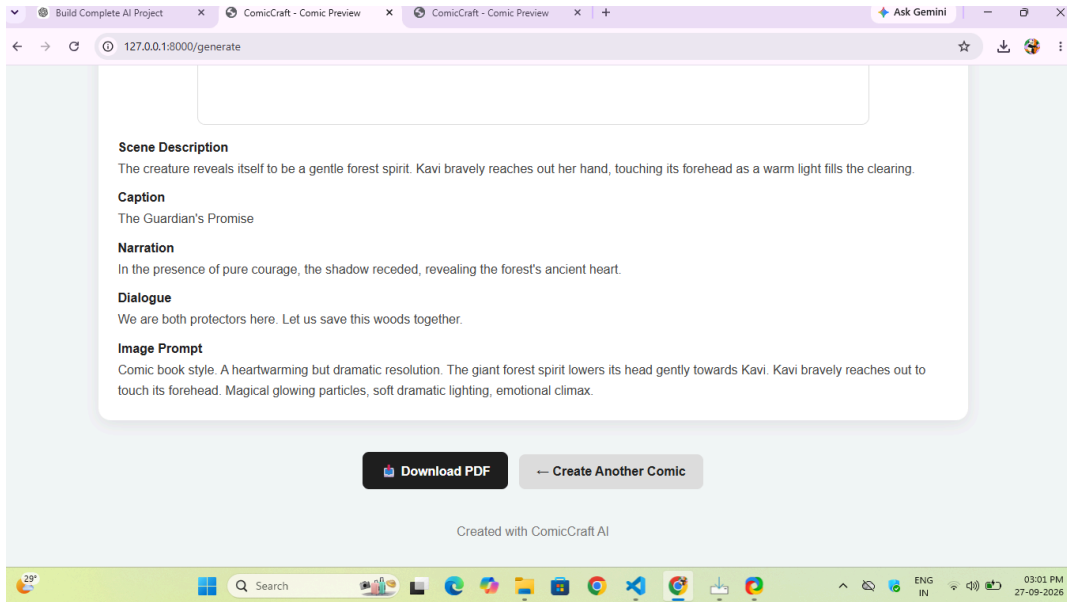
Any errors found during testing can be corrected before the final demonstration.



Input Page







Output Page

6.13 Conclusion

The Project Testing phase ensures that the developed ComicCraft AI application is checked from input to final output.

After testing, the project is prepared for **Phase 7 – Project Documentation**.

PHASE 7 – PROJECT DOCUMENTATION

7.1 Introduction

The Project Documentation phase records the development process, implementation details, testing information, and final project resources of ComicCraft AI.

Documentation helps explain how the project was developed and how the application can be understood and used.

7.2 Project Documentation Structure

The project documentation is organized according to the eight development phases:

1. Brainstorming and Ideation
2. Requirement Analysis
3. Project Design
4. Project Planning
5. Project Development
6. Project Testing
7. Project Documentation
8. Project Demonstration

This provides a clear record of the complete project lifecycle.

7.3 Source Code Documentation

The project contains separate files for different functions.

Important project files include:

- `main.py`
- `routes.py`
- `gemini_flash.py`
- `gemini_pro.py`
- `image_generator.py`
- `layout_builder.py`
- `exporters.py`

The project also contains frontend templates such as:

- `index.html`
- `comic_preview.html`
- `export_success.html`

The separation of files makes the project easier to understand and maintain.

7.4 Documentation of AI Components

The documentation records the purpose of each AI component.

Gemini Flash

Used for structured comic outline generation.

Gemini Pro

Used for detailed narration and dialogue generation.

Stable Diffusion

Used for comic-style image generation.

This separation explains how different AI capabilities contribute to the final comic.

7.5 Documentation of User Workflow

The documented user workflow is:

1. Open the ComicCraft AI application.
2. Enter a comic or story idea.
3. Enter the required character and story preferences.
4. Submit the request.
5. Wait for AI generation.
6. View the generated comic.
7. Review the comic panels.
8. Export the completed comic.

The project presentation also describes the basic user flow from entering an idea to displaying the generated output.

7.6 Project Repository

The source code and project-related files can be maintained in a GitHub repository.

The repository can contain:

- Source code
- Templates
- Static files
- Requirements
- Documentation
- Project phase files
- README
- Screenshots

7.7 Demonstration Materials

The project documentation can also include:

- Application screenshots
- Architecture diagram
- Workflow diagram
- Testing screenshots
- Final comic output
- GitHub repository link
- Demonstration video link

The project presentation provides placeholders for the GitHub repository and demonstration video.

7.8 Security and Configuration Documentation

Sensitive credentials such as API keys and tokens should be stored using environment variables rather than being written directly into source code.

This keeps configuration information separate from the main application code.

7.9 Conclusion

The Project Documentation phase provides a complete written record of ComicCraft AI.

It makes the project easier to understand, present, maintain, and evaluate.

The completed documentation is then used to support **Phase 8 – Project Demonstration**.

PHASE 8 – PROJECT DEMONSTRATION

8.1 Introduction

The Project Demonstration phase presents the completed ComicCraft AI application and explains how the system works from user input to final output.

The demonstration focuses on the actual working process of the application.

8.2 Demonstration Objectives

The main objectives of the demonstration are:

- To show the working application.
- To explain the user workflow.
- To demonstrate AI-based story generation.
- To display the generated comic.
- To demonstrate the final output.
- To explain the major technologies and components used.

8.3 Demonstration Procedure

Step 1 – Open the Application

The ComicCraft AI application is started using the local FastAPI server.

The application is opened in a web browser.

8.4 Step 2 – Enter the Comic Idea

The user enters a story or comic idea into the prompt field.

Additional preferences such as character, setting, tone, and art style can also be provided.

8.5 Step 3 – Submit the Request

The user submits the form.

The application sends the request to the FastAPI backend.

8.6 Step 4 – AI Story Generation

The backend processes the request and sends the relevant information to the AI generation components.

Gemini generates the structured story content according to the project workflow.

8.7 Step 5 – Image Generation

The image-generation component creates illustrations based on the generated scene prompts.

8.8 Step 6 – Comic Preview

The generated images and story content are organized into panels.

The comic preview displays the generated result in sequence.

8.9 Step 7 – PDF Export

After reviewing the generated comic, the user can select the PDF download option.

The system compiles the comic panels and related content into a PDF document.

8.10 Step 8 – Export Confirmation

After successful export, the application displays an export success page confirming the completed process.

8.11 Demonstration Flow

The complete demonstration can be represented as:

Open Application

↓

Enter Story Idea

↓

Submit Prompt

↓

AI Story Generation

↓

Panel and Image Generation

↓

Comic Preview

↓

PDF Export

↓

Export Confirmation

8.12 Expected Demonstration Output

The expected demonstration output is an AI-generated comic based on the user's idea.

The output may contain:

- Title
- Characters
- Scenes
- Panel structure
- Illustrations
- Narration
- Dialogues
- Final PDF

The project presentation describes the expected output as a generated comic story based on the user's idea, with elements such as title, characters, scenes, and dialogues depending on the implementation.

8.13 Benefits Demonstrated

The project demonstrates the practical use of Generative AI for creative content generation.

The application can:

- Reduce effort during story ideation
- Help beginners create structured stories
- Demonstrate practical AI usage
- Encourage creative experimentation
- Provide an AI-assisted storytelling workflow

8.14 Future Enhancements

Possible future enhancements include:

- More advanced character customization
- Multiple storytelling styles
- User accounts
- Comic libraries
- Longer multi-page story arcs
- Voice narration

- Multilingual support
- Additional customization options

The project documentation also identifies future improvements such as character customization, multiple storytelling styles, saving/downloading comics, voice narration, and multilingual support.

8.15 Final Conclusion

ComicCraft AI demonstrates how Artificial Intelligence can be integrated with web development to create an AI-assisted comic generation application.

The project provides a complete workflow starting from a user's creative idea and continuing through story generation, panel creation, illustration generation, comic layout, preview, and PDF export.

The project combines AI, software development, web technologies, and creative storytelling in a single application.

Therefore, ComicCraft AI provides a practical demonstration of how Generative AI can be used to simplify and support creative content creation.
